МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
Ордена Трудового Красного Знамени
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Лабораторная работа №2

по дисциплине «Структуры и алгоритмы обработки данных»
по теме «Деревья и быстрые запросы к данным»

Выполнил студент группы:
БПИ2401
Еланская Екатерина Дмитриевна
Проверил: Борзов В.М.

Москва, 2026

Оглавление

Введение **3**

Выполнение **4**
Задание 1. Создание бинарного дерева 4
Задание 2. Вставка в бинарное дерево поиска 4
Задание 3. Поиск элемента 4
Задание 4. Обходы дерева 5
Задание 5. Удаление из BST 5
Задание 6. Влияние порядка вставки на высоту дерева 6
Задание 7. Повороты AVL-дерева 6
Задание 8. Наивный запрос суммы диапазона 7
Задание 9. Префиксные суммы 8
Задание 10. Дерево Фенвика 8
Задание 11. Дерево отрезков для суммы 9
Задание 12. Сумма, минимум и максимум 10
Задание 13. Самостоятельная работа 11

Вывод **13**

Контрольные вопросы **14**

Введение

Целью данной лабораторной работы является:

— изучение применения древовидных структур данных для быстрого поиска и обработки запросов;

— освоение операций с бинарным деревом поиска, понимание влияния высоты дерева на сложность алгоритмов и принцип балансировки;

— получение практических навыков использования префиксных сумм, дерева Фенвика и дерева отрезков для эффективной обработки запросов к диапазонам данных.

Задачи:

1) реализовать бинарное дерево поиска и основные операции с ним;

2) исследовать влияние порядка вставки на высоту дерева;

3) познакомиться с балансировкой на примере AVL-дерева;

4) сравнить несколько подходов к вычислению диапазонных запросов и точечных обновлений.

Выполнение

Задание 1. Создание бинарного дерева

Вывод бинарного дерева представлен в листинге 1. Исходный код представлен в листинге 2.

```
root=8, right_child=12, leaf=14
```

Листинг 1 — Вывод элементов бинарного дерева

```
class Node:
    def __init__(self, val, il=None, ir=None):
        self.key = val
        self.left = il
        self.right = ir

    def __str__(self):
        return f"{self.key}"

    def __repr__(self):
        return self.__str__()

bin_tree = Node(
    8,
    Node(4, Node(2), Node(6)),
    Node(12, Node(10), Node(14))
)
```

Листинг 2 — Создание бинарного дерева

Задание 2. Вставка в бинарное дерево поиска

Вывод бинарного дерева, созданного с помощью функции вставки, представлен в листинге 3. Исходный код представлен в листинге 4.

```
root=8, right_child=4, leaf=6
```

Листинг 3 — Вывод элементов бинарного дерева, созданного с помощью функции вставки

```
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
```

Листинг 4 — Функция вставки в бинарное дерево

Задание 3. Поиск элемента

Вывод найденного элемента бинарного дерева представлен в листинге 5. Исходный код представлен в листинге 6.

```
near root: (8, 1)
outer level: (14, 3)
not existing: (None, 3)
```

Листинг 5 — Вывод найденного элемента

```
def search(root, key):
    visited = 0
    current = root
    while current is not None:
        visited += 1
        if key == current.key:
            return current, visited
        if key < current.key:
            current = current.left
        else:
            current = current.right
    return None, visited
```

Листинг 6 — Функция поиска в бинарном дереве

Задание 4. Обходы дерева

Вывод бинарного дерева в порядке обхода представлен в листинге 7. Исходный код представлен в листинге 8.

```
preorder: [8, 4, 2, 6, 12, 10, 14]
inorder: [2, 4, 6, 8, 10, 12, 14]
postorder: [2, 6, 4, 10, 14, 12, 8]
```

Листинг 7 — Вывод бинарного дерева в порядке обхода

```
def preorder(root):
    if root is not None:
        return [root.key] + preorder(root.left) + preorder(root.right)
    return []

def inorder(root):
    if root is not None:
        return inorder(root.left) + [root.key] + inorder(root.right)
    return []

def postorder(root):
    if root is not None:
        return postorder(root.left) + postorder(root.right) + [root.key]
    return []
```

Листинг 8 — Функции вывода бинарного дерева

Задание 5. Удаление из BST

Вывод удаления элемента из бинарного дерева представлен в листинге 9. Исходный код представлен в листинге 10.

```
removed leaf: [2, 4, 6, 8, 10, 12]
removed element with single child: [8, 4, 2, 6, 10]
removed element with two children: [8, 6, 2, 10]
```

Листинг 9 — Вывод удаления из бинарного дерева

```
def delete(root, key):
    if root is None:
        return None
    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        min_node = root.right
        while min_node.left is not None:
            min_node = min_node.left
        root.key = min_node.key
        root.right = delete(root.right, min_node.key)
    return root
```

Листинг 10 — Функция удаления из бинарного дерева

Задание 6. Влияние порядка вставки на высоту дерева

Вывод бинарных деревьев, различающихся порядком вставки, представлен в листинге 11. Исходный код представлен в листинге 12.

```
height of [8, 4, 2, 6, 12, 10, 14] is 2
height of [2, 4, 6, 8, 10, 12, 14] is 6
```

Листинг 11 — Вывод бинарных деревьев, различающихся порядком вставки

```
def height(root):
    if root is not None:
        return 1 + max(height(root.left), height(root.right))
    return -1
```

Листинг 12 — Вывод бинарных деревьев и функция определения их высоты

Задание 7. Повороты AVL-дерева

Вывод поворотов и балансировки бинарного дерева представлен в листинге 13. Исходный код представлен в листинге 14.

```
balanced [30, 20, 10] is [20, 10, 30]
balanced [10, 20, 30] is [20, 10, 30]

double rotation of [30, 10, 20] is [20, 10, 30]
double rotation of [10, 30, 20] is [20, 10, 30]
```

Листинг 13 — Вывод поворотов и балансировки бинарного дерева

```
def node_height(node):
    if node is not None:
```

```
        return 1 + max(node_height(node.left), node_height(node.right))
    return -1

def balance_factor(node):
    if node is not None:
        return node_height(node.left) - node_height(node.right)
    return 0

def rotate_right(root):
    new_root = root.left
    root.left = new_root.right
    new_root.right = root
    return new_root

def rotate_left(root):
    new_root = root.right
    root.right = new_root.left
    new_root.left = root
    return new_root

def balance(node, key):
    if node is None:
        return Node(key)
    if key < node.key:
        node.left = balance(node.left, key)
    elif key > node.key:
        node.right = balance(node.right, key)
    else:
        return node

    bf = balance_factor(node)
    # LL
    if bf > 1 and key < node.left.key:
        return rotate_right(node)
    # RR
    if bf < -1 and key > node.right.key:
        return rotate_left(node)
    # LR
    if bf > 1 and key > node.left.key:
        node.left = rotate_left(node.left)
        return rotate_right(node)
    # RL
    if bf < -1 and key < node.right.key:
        node.right = rotate_right(node.right)
        return rotate_left(node)
    return node
```

Листинг 14 — Функции поворота и балансировки

Задание 8. Наивный запрос суммы диапазона

Вывод наивной суммы диапазонов бинарного дерева представлен в листинге 15. Исходный код представлен в листинге 16.

```
range_sum_naive elapsed=0.7601s
```

Листинг 15 — Вывод наивной суммы диапазонов бинарного дерева

```
def range_sum_naive(data, left, right):
```

```
    s = 0
    for i in range(left, right + 1):
        s += data[i]
    return s

K = 1000
ranges = []
for k in range(K):
    l = randint(0, BIG_NUM - 1)
    ranges.append(
        [l, randint(l, BIG_NUM - 1)]
    )
```

Листинг 16 — Наивный подсчет суммы

Задание 9. Префиксные суммы

Вывод префиксной суммы бинарного дерева представлен в листинге 17. Исходный код представлен в листинге 18.

```
build_prefix elapsed=0.0068s
range_sum_prefix elapsed=0.0003s
```

Листинг 17 — Вывод префиксной суммы бинарного дерева

```
def build_prefix(data):
    prefix = [0] * (len(data) + 1)
    for i, val in enumerate(data):
        prefix[i + 1] = prefix[i] + val
    return prefix

def range_sum_prefix(prefix, left, right):
    return prefix[right + 1] - prefix[left]
```

Листинг 18 — Подсчет префиксной суммы

Задание 10. Дерево Фенвика

Вывод дерева Фенвика и его операций представлен в листинге 19. Исходный код представлен в листинге 20.

```
elapsed fenwick range_sum=1208820s, sum=1208820s for range[66998, 91007]
elapsed fenwick range_sum=929396s, sum=929396s for range[9642, 27989]
elapsed fenwick range_sum=14142s, sum=14142s for range[98345, 98603]
elapsed fenwick range_sum=113434s, sum=113434s for range[68688, 70956]
elapsed fenwick range_sum=1018573s, sum=1018573s for range[49169, 69335]
```

Листинг 19 — Вывод дерева Фенвика

```
class FenwickTree:
    def __init__(self, data=None):
        if data:
            self.n = len(data)
        else:
            self.n = 0
        self.tree = [0] * (self.n + 1)
        if data:
```

```
            for i, val in enumerate(data):
                self.update(i, val)

    def update(self, index, delta):
        i = index + 1
        while i <= self.n:
            self.tree[i] += delta
            i += i & -i

    def prefix_sum(self, index):
        i = index + 1
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & -i
        return s

    def range_sum(self, left, right):
        if left == 0:
            return self.prefix_sum(right)
        return self.prefix_sum(right) - self.prefix_sum(left - 1)
```

Листинг 20 — Класс дерева Фенвика

Задание 11. Дерево отрезков для суммы

Вывод дерева отрезков и его операци представлен в листинге 21. Исходный код представлен в листинге 22.

```
seg_tree=1208820 for range[66998, 91007]
seg_tree=929396 for range[9642, 27989]
seg_tree=14142 for range[98345, 98603]
seg_tree=113434 for range[68688, 70956]
seg_tree=1018573 for range[49169, 69335]
```

Листинг 21 — Вывод дерева отрезков

```
class SegmentTree:
    def __init__(self, data):
        n = len(data)
        self.n = n
        self.data = data
        self.tree = [0] * (2 * self.n)
        for i in range(n):
            self.tree[n + i] = data[i]
        for i in range(n - 1, 0, -1):
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def query(self, left, right):
        res = 0
        l = left + self.n
        r = right + self.n
        while l <= r:
            if l & 1:
                res += self.tree[l]
                l += 1
            if not (r & 1):
                res += self.tree[r]
                r -= 1
```

```
            l //= 2
            r //= 2
        return res

    def update(self, index, value):
        self.data[index] = value
        i = index + self.n
        self.tree[i] = value
        i //= 2
        while i >= 1:
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]
            i //= 2
```

Листинг 22 — Класс дерева отрезков

Задание 12. Сумма, минимум и максимум

Вывод поиска суммы, минимума и максимума представлен в листинге 23. Исходный код представлен в листинге 24.

```
seg_tree_mms=(1208820, 1, 100) for range[66998, 91007]
seg_tree_mms=(929396, 1, 100) for range[9642, 27989]
seg_tree_mms=(14142, 2, 100) for range[98345, 98603]
seg_tree_mms=(113434, 1, 100) for range[68688, 70956]
seg_tree_mms=(1018573, 1, 100) for range[49169, 69335]
```

Листинг 23 — Вывод суммы, минимума и максимума дополненного дерева отрезков

```
class SegmentTreeMMS(SegmentTree):
    NEUTRAL = (0, float('inf'), float('-inf'))

    def __init__(self, data):
        super().__init__(data)
        n = self.n
        for i in range(n):
            val = data[i]
            self.tree[n + i] = (val, val, val)
        for i in range(n - 1, 0, -1):
            a = self.tree[2 * i]
            b = self.tree[2 * i + 1]
            self.tree[i] = (
                a[0] + b[0],
                min(a[1], b[1]),
                max(a[2], b[2])
            )

    def query(self, left, right):
        res_l = self.NEUTRAL
        res_r = self.NEUTRAL
        l = left + self.n
        r = right + self.n
        while l <= r:
            if l & 1:
                a = res_l
                b = self.tree[l]
                res_l = (
                    a[0] + b[0],
                    min(a[1], b[1]),
```

```
                    max(a[2], b[2])
                )
                l += 1
            if not (r & 1):
                a = self.tree[r]
                b = res_r
                res_r = (
                    a[0] + b[0],
                    min(a[1], b[1]),
                    max(a[2], b[2])
                )
                r -= 1
            l //= 2
            r //= 2
            a = res_l
            b = res_r
        return (
            a[0] + b[0],
            min(a[1], b[1]),
            max(a[2], b[2])
        )

    def update(self, index, value):
        self.data[index] = value
        i = index + self.n
        self.tree[i] = (value, value, value)
        i //= 2
        while i >= 1:
            a = self.tree[2 * i]
            b = self.tree[2 * i + 1]
            self.tree[i] = (
                a[0] + b[0],
                min(a[1], b[1]),
                max(a[2], b[2])
            )
            i //= 2
```

Листинг 24 — Дополненный класс дерева отрезков

Задание 13. Самостоятельная работа

Код программы для анализа последовательности измерений датчика представлен в листинге 25. Вывод программы представлен в листинге 26.

```
K = 1000
sensor_ids = list(set([randint(1, 100) for i in range(20)]))
sensor_tree = None
sensors = dict()
for sid in sensor_ids:
    data = [randint(0, 100) for i in range(K)]
    sensors[sid] = {
        'sid': sid,
        'data': data,
        'fen_tree': FenwickTree(data),
        'seg_tree': SegmentTreeMMS(data)
    }
    sensor_tree = insert(sensor_tree, sid)
print('sid_tree={}'.format(inorder(sensor_tree)))
sid = sensor_ids[5]
```

```
l = 100
r = 300
print('for sid={} range_sum_naive={}, range_sum={}'.format(
    sid,
    range_sum_naive(sensor_tree[sid].data, l, r),
    sensor_tree[sid].fen_tree(l, r),
))
```

Листинг 25 — Программа для анализа последовательности измерений датчика

```
```

Листинг 26 — Вывод программы

Вывод

В ходе лабораторной работы были:

— изучены применения древовидных структур данных для быстрого поиска и обработки запросов;

— освоены операции с бинарным деревом поиска, получено понимание влияния высоты дерева на сложность алгоритмов и принцип балансировки;

— получены практические навыки использования префиксных сумм, дерева Фенвика и дерева отрезков для эффективной обработки запросов к диапазонам данных.

Контрольные вопросы

1) *Что называется деревом и какие основные элементы используются при его описании?*
Дерево – иерархическая структура из узлов и связей между ними. Основные элементы: корень, родитель, потомок, лист, поддерево, глубина узла и высота дерева.

2) *Что такое высота дерева и как она влияет на стоимость операций?*
Высота h – максимальное число ребер от корня до листа. Поиск, вставка и удаление в BST выполняются за $O(h)$, поэтому меньшая высота дает логарифмическую сложность вместо линейной.

3) *Чем бинарное дерево поиска отличается от произвольного бинарного дерева?*
В BST действует правило упорядочения: все ключи левого поддерева меньше ключа узла, а правого – больше. Произвольное бинарное дерево такого порядка не задает, и быстрый поиск в нем невозможен.

4) *Как выполняются поиск и вставка ключа в BST?*
Оба алгоритма спускаются от корня, сравнивая ключ с текущим узлом и уходя влево или вправо. Поиск останавливается при совпадении или на пустой ссылке, а вставка помещает новый узел в первую пустую позицию.

5) *Какие случаи необходимо учитывать при удалении узла из BST?*
Три случая: удаление листа, удаление узла с одним потомком (потомок занимает его место) и удаление узла с двумя потомками (ключ заменяется минимумом правого или максимумом левого поддерева).

6) *Чем отличаются preorder, inorder и postorder?*
Порядком посещения узла относительно поддеревьев: preorder – узел, левое, правое; inorder – левое, узел, правое; postorder – левое, правое, узел.

7) *Почему inorder-обход BST возвращает ключи в отсортированном порядке?*
Потому что сначала выводятся все меньшие ключи левого поддерева, затем сам узел, затем все большие ключи правого. Конкатенация этих частей дает возрастающую последовательность.

8) *Почему BST может выродиться при неудачном порядке вставки?*
При вставке отсортированных значений каждый новый ключ становится правым (или левым) потомком предыдущего, и дерево становится высоты $n - 1$ со сложностью операций $O(n)$.

9) *Что такое баланс-фактор AVL-узла?*
Это разность высот левого и правого поддеревьев узла. В корректном AVL-дереве она принимает значения -1, 0 или 1.

10) *Для чего используются одинарные и двойные повороты AVL-дерева?*
Для восстановления баланса после вставки или удаления. Одинарные повороты исправляют LL- и RR-случаи, двойные – LR- и RL-случаи.

11) *Чем в общем виде красно-черное дерево отличается от AVL-дерева?*
Красно-черное дерево балансируется менее строго (через цвета узлов), поэтому его высота может быть больше, но поворотов при вставке и удалении обычно меньше.

12) *Почему B-Tree и B+Tree удобны для внешней памяти?*
В одном узле хранится много ключей, поэтому дерево получается низким и число обращений к диску (чтений блоков) резко сокращается.

13) *В чем основная особенность организации данных в B+Tree?*
Все записи хранятся в листьях, а внутренние узлы используются только для навигации; листья связаны между собой, что удобно для последовательного чтения диапазонов.

14) *Что называется диапазонным запросом и зачем нужна предварительная обработка?*
Диапазонный запрос – вычисление характеристики (суммы, минимума, максимума) на отрезке массива. Предварительная обработка нужна, чтобы отвечать на множество запросов быстрее, чем заново перебирать элементы.

15) *Как строятся префиксные суммы и как с их помощью получить сумму диапазона?*
Строится массив prefix, где prefix[i + 1] = prefix[i] + data[i]. Тогда сумма на [left, right] равна prefix[right + 1] - prefix[left].

16) *Почему префиксные суммы неудобны при частых изменениях массива?*
После изменения одного элемента приходится пересчитывать все последующие префиксы, что в худшем случае требует $O(n)$ операций на каждое обновление.

17) *Для чего используется дерево Фенвика?*
Для быстрого вычисления префиксных сумм и точечных обновлений – обе операции выполняются за $O(log_n)$ при $O(n)$ памяти.

18) *Что определяет выражение i & -i в дереве Фенвика?*
Младший установленный бит числа i, то есть размер блока, за который

отвечает узел с индексом i. Именно на эту величину меняется индекс при подъеме и спуске по дереву.

19) *Как получить сумму диапазона с помощью дерева Фенвика?*
Как разность двух префиксных сумм: pref_sum[right] - pref_sum[left-1].

20) *Что хранит узел дерева отрезков?*
Агрегированную характеристику своего диапазона: сумму, минимум, максимум или другую величину, которую можно объединить из значений дочерних узлов.

21) *Как выполняются диапазонный запрос и точечное обновление дерева отрезков?*
Запрос рекурсивно спускается по дереву, полностью используя узлы, целиком попавшие в диапазон, и дробя частично пересекающиеся. Обновление меняет лист и пересчитывает всех его предков до корня; обе операции – $O(log_n)$.

22) *В каких случаях удобнее применять префиксные суммы, дерево Фенвика и дерево отрезков?*
Префиксные суммы – при статическом массиве и частых запросах. Дерево Фенвика – при частых точечных обновлениях и запросах суммы. Дерево отрезков – когда нужны произвольные агрегаты (минимум, максимум) и обновления.